Staying Hands-On, as an Engineering Manager or a Tech Lead

The importance of onboarding like a software engineer, activities to stay hands-on, and the reality of writing code.

GERGELY OROSZ

FEB 14, 2023 • PAID

> ## Q: “As an engineering manager/tech lead, how can I stay hands-on and technical, when I’m no longer coding on a regular basis?”

I’ve been getting variations of this question more frequently in recent months. It’s a common query among engineers who are becoming engineering managers or tech-lead managers, who worry their skills will grow rusty as they spend less time coding.

I also sense a renewed focus on this topic by engineering managers who want to ensure they are close enough to where the work is done, and to keep their employment prospects bright should they want to change jobs internally, or search for a new employer. Being a hands-on engineering leader can only be a positive, and of course most engineering manager interview processes include technical interviews, as we cover in the issue Hiring engineering managers.

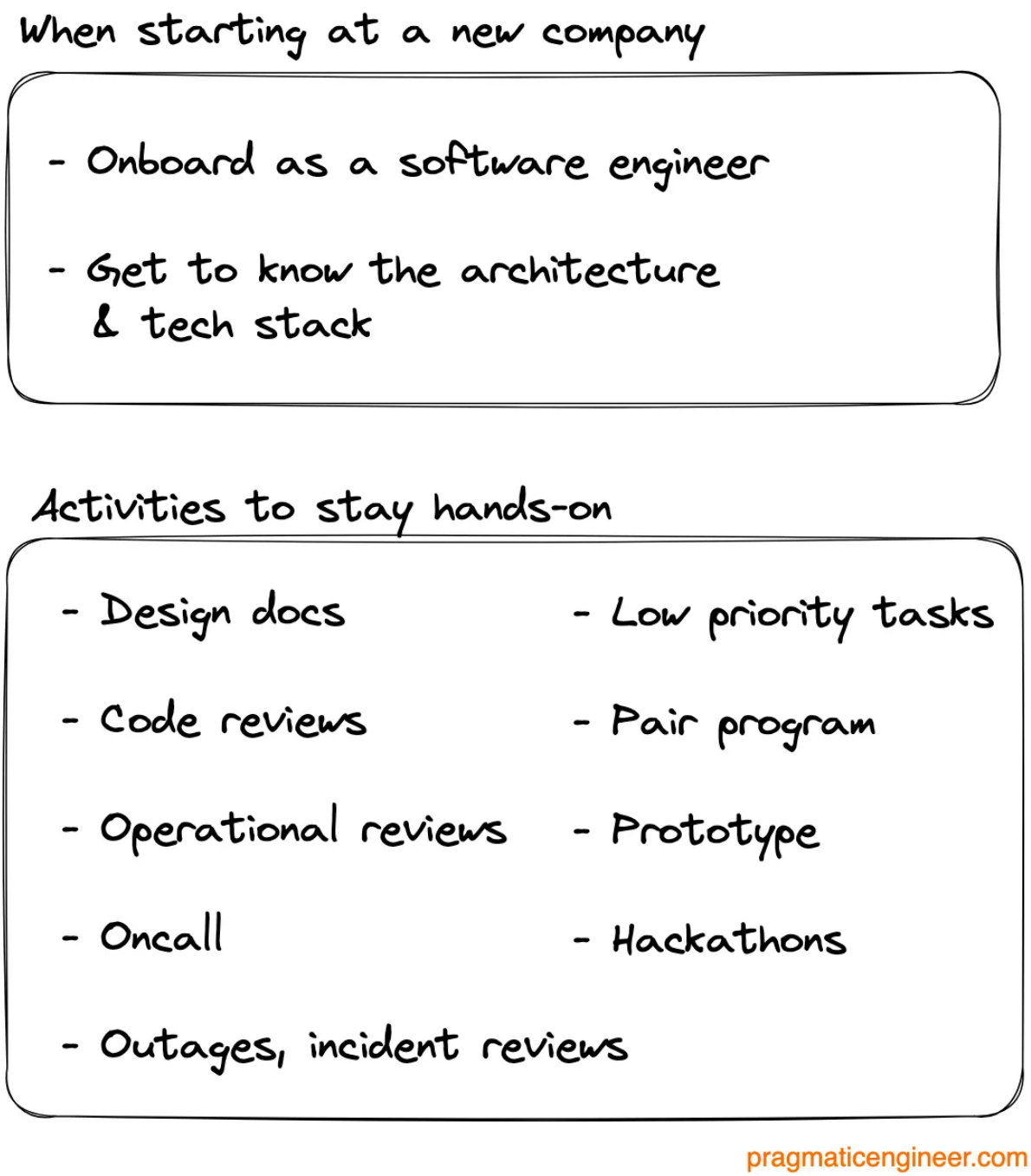
Just last week, we covered the emerging trend where we could be seeing fewer middle managers across tech companies. If this trend is to continue, managers and tech leads who are not hands-on could have a harder time switching jobs. And it’s not just engineering managers: I recently talked with a VP of Engineering who was offered a Head of Engineering role at a scaleup with 15 engineers, and this offer was extended after the CEO was satisfied on how this VPE felt like they are able to be hands-on, when needed.

Being “hands-on” and “technical enough” are different, though. Today, we cover being hands-on, and will tackle “staying technical” in a follow-up issue. In this issue, we examine:

1. Why it’s important to stay hands-on
2. Setting yourself up for success, from the start
3. Activities to stay hands-on with
4. Writing code
5. Beware of taking space from others
6. The reality of staying hands-on

While this issue is written with engineering managers in mind, other professionals in roles where it’s hard to set aside time for coding, can benefit from these approaches. Specifically, tech leads and staff+ engineers – who have many other priorities – may see value in the approaches below.

We will cover how to stay technical - even when no longer being hands-on - in a separate newsletter issue.



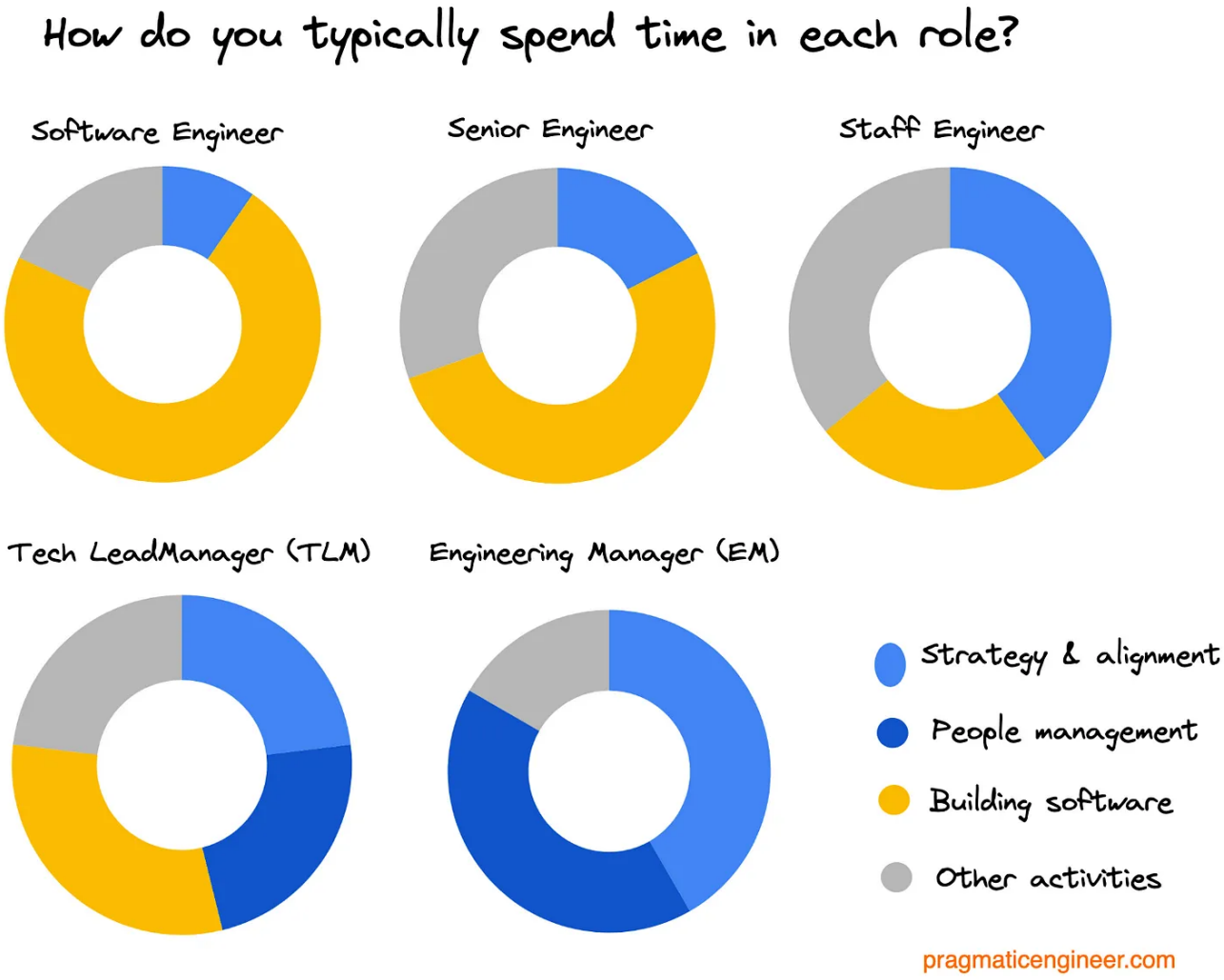
Some of the activities to stay hands-on, both when starting at a new company, and activities that can help staying hands-on.

1. Why it's important to stay hands-on and technical

Being hands-on and being technical are separate but related concepts. So let's split them out.

Being hands-on means rolling up your sleeves and doing the actual work. As an engineering manager or a tech lead, this means doing the planning, writing the code, doing the code review, deploying, being oncall, and other activities.

Once you move on from being a software engineer to being a tech lead manager or engineering manager, then your focus and priorities change. Doing hands-on work like coding is no longer your top priority. Instead, empowering your team to execute well and work on the right tasks becomes more important. Where you spend your time and focus changes, as we cover in the article, Engineering skill set overlaps.



Engineering skill set overlaps between software engineer, TLM and EM roles. TLM and EM roles introduce management and other responsibilities which take time and focus away from hands-on building of software.

Thanks to the added responsiblities, being hands-on as a manager is more tricky; you have the authority of your position and are responsible for your team's performance. If you change nothing in your working style, for example by continuing to claim the most challenging tickets, or by doing the lion's share of code reviews, then your behavior risks coming across as micro-managing, or like you're hoarding the most impactful work; those tasks which could help engineers on your team to demonstrate impact, grow professionally, and perhaps aid their promotion cases.

But even if you want to, it's very hard to be as hands-on as a manager as you were as an engineer, due to lack of time. Fortunately, being incredibly hands-on is not the only way to stay connected to your team – or the wider tech industry.

Staying technical and involved is a less time-intensive alternative for keeping close to the action. Instead of being hands-on in the literal sense of doing the team's tasks, you can oversee their work and stay in the know about what's happening.

What are the benefits of being hands-on, as a manager? There are several:

- *Trust with the team.* Engineers tend to trust managers who are visibly hands-on.
- *Easier to stay up to date*. By being hands-on, you keep pace with the technologies your team works with.
- *Keeping career paths open*. As a hands-on lead or engineering manager, you always have the option of pivoting to being an individual contributor. This means more options when you're hunting a new job, as there are usually several times more engineering vacancies than manager positions. Also, there are usually about the same number of Staff+ positions as manager positions. So being a hands-on manager could double the amount of viable options in the job market.

2. Setting yourself up for success, from the start

When you join a company as a manager or a lead, there are a few things you can do to make it easier to quickly become hands-on, and to stay hands-on.

Onboard to your team like a software engineer. When joining a new company as a manager or lead, you'll feel the temptation to get involved with managerial work right

away: meeting stakeholders, 1:1s with engineers, putting out fires. The list goes on.

Resist the temptation to devote all your energy to managerial work, and instead carve out time to go through the engineering onboarding process. Set up your local development environment and complete the same onboarding tasks as software engineers do when they join your team.

Some companies make this type of "managers working as engineers" onboarding process mandatory. A well-known case is Meta, where managers go through the same 6-7 week-long engineering bootcamp as engineers.

Some startups also adopt this mindset. When mobile subscriptions platform RevenueCat was hiring its first engineering manager, it set expectations that the successful candidate would spend their first month partnering with an engineer and seeing a project through to completion. They only started managing a team after six months in the role.

Doing the work engineers do is the best way to be hands-on and stay technical in the workplace. If you skip this, you'll be playing catch-up later and will lack empathy for what the developer workflow in your team is, and how things are connected.

Get to know your team's architecture and technology stack inside out, and keep up to date with it. As a manager on your team, there's little excuse for not knowing how everything works and which technologies are used and why.

A rule I learned is this: you can't prove you know how something works until you can picture it for others and teach it to them. Conveniently enough, the onboarding of new joiners is a responsibility that ultimately falls on you, as a manager. So lean in and create – or improve – onboarding materials for the next new engineer and volunteer to onboard them to the technology stack.

If you're surprised by this idea: ask yourself, why? Are you not familiar enough with the systems to explain how they work and convey their underlying reasoning and logic? If not, you have catching up to do with how your team works. And if you do know, then explaining it to a new joiner just confirms it.

Consider creating a "team introduction" document or presentation for capturing technical details. Another handy method I've found for keeping up with the technology choices of my team, is to create a "team introduction" slideshow as a brief summary of the team when meeting managers of other teams.

This introduction document contains:

- The vision and mission of our team
- Areas we own
- Overview of the backend architecture: systems, technologies, how they link up
- Overview of the frontend architecture
- Product roadmap
- Engineering roadmap: upcoming work related to technology changes, migrations, improvements

Personally, I was surprised by just how much I learned from putting this document together, and pairing with engineers to ensure I understood the details of our architecture and technology choices. Also, this document needs to be kept up to date by its author – the engineering manager! – which is another good reason for keeping track of team-level technology and systems changes.

3. Activities to stay hands-on with

What are the activities which it's useful to be hands-on with, as a manager? Here's the most common ones in my experience, and some which managers have shared. Keep in mind, there's a balance to strike in how much attention you give to particular activities, and we'll cover more on this in the 'Beware of taking up too much space' section.

Design docs / RFCs / ERDs / ARDs. If your organization writes and circulates design documents, one of the best ways to be hands-on is to get actively involved with these documents. Don't just read them; comment on them. Challenge decisions, ask for clarifications, offer alternative approaches. If your company uses the concept of approvers, then become an approver for some plans and ensure you provide timely feedback.

At companies which make heavy use of these documents, the biggest challenge is the time it takes to review them and give quality feedback. My advice is to regularly set aside time for this, and always prioritize documents involving your team.

When joining a company that uses these documents, they are a fantastic source of learning. For example, when I joined Uber, I made a habit of reading RFCs, making notes of concepts I did not understand and either looking them up myself, or asking team members to explain them.

Documents are covered in depth in Engineering planning with RFCs, design documents and ADRs.

Code reviews. Even if you don't write code, you can still review it. However, if you have a busy schedule with less time for code reviews, be clear about whether or not your comments are blocking; that is, can a change be merged before they're addressed, or not? The problem with a blocking comment is that if someone disagrees, they might need to discuss it with you. But if you're on a manager's schedule with lots of meetings, your availability may be poor and in this way, you could slow down coding.

You could also block out specific times for doing code reviews. Another alternative is to ensure you are not the primary reviewer, and therefore won't block code changes from being merged.

Operational reviews. Some teams regularly review their operational metrics, such as systems' health metrics, the operational dashboard and other indicators of how products and systems are performing.

Amazon calls its version of this activity, the "Wednesday Ops meeting." As we cover in Inside Amazon's Engineering Culture:

> "The Wednesday Ops meeting is a weekly meeting to which each service team must send one delegate. Usually, directors and VPs of service teams are there, as well. Op wins are reviewed, teams present past CoEs and then 'spin the wheel'. Each service team is on the wheel and the "winner" gets to present their services' operational dashboard to the room.

> These meetings are usually preceded by team and org-level ops meetings, which then feed into the AWS-level meeting. When a team "wins" the wheel spin, they're expected to lead attendees through the dashboards they themselves review on a weekly basis."

If your team has such a meeting, facilitating it is a good way to stay closely involved and also to learn when to take a more hands-on approach by investigating a problematic metric or area. If there is no such meeting, should there be one for you and engineers to review the team's health metrics?

When the data indicates there's an issue, you can help investigate the cause and what should be done in response.

Mark Tsimelzon is an engineering director at WhatsApp with seven recommendations for staying technical as an engineering manager. As he writes:

> "Metrics is another great area to understand. Understanding data helps you to participate in goal setting for the team. Also, some of the details of what a given piece of data means, and where it comes from, can be very technically involved. You can help your team by becoming an expert on this, and it's a good way to keep being exposed to some technical issues."

Go oncall. If your team has an oncall rotation, volunteer to take part of a shift, or a full shift. Being oncall is very rarely a responsibility engineers *want*; it's more of a *must* do. Therefore any help to lighten the load will be appreciated.

To go oncall, you need to be up to speed on how to investigate issues, review logs and debug parts of the system. Being oncall also helps you understand what the experience is like for engineers. How often do alerts come in? How many of the alerts are noisy and which ones indicate genuine outages? How stressful is being oncall?

When I worked at Uber, one of the best things I did as a manager was to occasionally go oncall. Previously, I *kind of* knew about the challenges of being oncall. However, knowing that people get paged in the dead of night, and actually *being* the one getting paged, then waking up and attempting to debug the issue – and perhaps struggling to – was a very different and eye-opening experience. After I went through a poor oncall

myself, I made it a point of principle to ensure we had ample space to resolve oncall reliability issues, and to pause other work while our oncall system was noisy enough to wake up engineers at night.

For approaches on doing oncall better, see the issue Healthy oncall practices.

Outages and incident reviews. As a manager of a team, you're usually expected to know what happens when things go wrong. It can also help your team if you provide cover when needed, and escalate to other teams if it helps mitigate an outage, or follow up on it.

Consider directing your team members to ping you about outages that are medium or high priority, so you're aware of them, and – if it makes sense – can help with mitigation. Of course, as a manager, pay attention to team dynamics. For example, if an incident already has a commander, then it could be poor form to take over command of the incident, without making this change of roles clear. Also bear in mind that taking the lead might encourage team members to rely on you to handle incidents, when perhaps this isn't your goal.

Once an outage is mitigated, be sure to understand its causes. Incident write-ups and investigations are among the more thankless tasks, so taking full ownership of a postmortem can be a great way to both be hands-on, to learn about the systems, do more digging within logs, and to lead by example on what good postmortem documents and follow-up work looks like.

It may seem far-fetched that you'd want to lead postmortems and the clean-up work every time, but I'd still recommend doing so at least once as a manager. You can consider leading the incident review if it's helpful for the team. This might be the case for high-profile incidents when doing so shows accountability for the team, by personally tackling hard questions arising from the review.

At companies like Amazon, there's no question that managers are part of incident reviews – or SEV reviews as the online retail giant calls them. I suggest following a similar approach. Leadership when things go wrong is especially important, like during and after an outage. Also, outages are one of the few events for which there's not many

excuses for a team manager to lack all the information about why it happened, and what will be done to avoid a repeat in future.

At larger companies, there could be a rotation among managers and senior engineers for leading outage reviews. A great way to stay hands-on and also learn more about outages, is to volunteer to facilitate these meetings, with you being the one asking tough questions to uncover the root causes of an outage and identifying the work needed to make the systems more resilient.

Read more about this topic in Incident review and postmortem best practices.

4. Writing code

I've left the most obvious activity for staying hands-on until the end: do some coding! Of course, this is often easier said than done. The challenge of following this advice is that as a manager, you have much less uninterrupted time. If you pick up a coding task, there's a good chance you'll be interrupted and won't get it done in time. This makes it unwise to pick up any time-sensitive, critical coding task.

Here are some things you can do, though:

Work on low-priority tasks. Pick up improvements or bugs the team wouldn't get to, which aren't a problem if not done. Choose tasks that are reasonable effort, like simple fixes or interesting bugs, where you can explore new parts the system as you debug them.

The nice thing about doing even very small tasks is that you acquire a feel for the developer workflow and understand the pain points. If you don't code often, then doing a coding task every now and then helps to refamiliarize yourself with the environment, and notice tooling changes that have happened in the meantime.

Pair program. Pair with another engineer on the team to work on a task. This could mean helping them out by being more of the secondary, or by leading the session.

Prototype or build a throwaway proof of concept. Prototyping can be a great way to both be hands-on, and also help your team. However, be aware that by building the first

prototype of an approach, you might impose design decisions on the team which they feel obliged to follow, as they don't want to go against your approach due to your seniority. So make sure your prototyping does not disempower engineers from making their own decisions, or ones that are different from yours, if more practical.

Join hackathons and 'fixit weeks.' If your team dedicates uninterrupted time to things like a hackathon or a 'fixit week' where they fix tech debt, consider clearing your calendar and joining in. These events are typically few and far between, and can be a great way to get hands-on.

Finding the time for coding - in a way that is hands-on and helps the team - is the biggest obstacle to doing it. Charlie Roman is a technical director at the indie games studio, Player1stGames, and sets aside dedicated time for coding:

Charlie Roman
@CharlesRoman

@GergelyOrosz Personally, no matter how busy I get with other things, I like to reserve 5-10% of my time for coding tasks. In addition, I will review TDDs and do code reviews. I find it helps me be a better manager.

8:43 PM · Jan 29, 2023

The reality of writing code that is helpful for the team is how this type of is usually less helpful for the team when you're a manager. As a manager, or lead, your goal is not to act as yet another senior software engineer who churns out code: but it's to multiply the team's output by doing things that are *not* you coding by yourself. While it can feel good to retreat and write code - and enjoy your code doing what it should be doing - beware of spending too much time on these activities, while neglecting the things that your team *really* needs from you.

Just how important - or helpful - for you to write code depends on your team's size, seniority, and dynamics, of course. If you're more of a tech lead manager (TLM) with few reports - most of whom are senior - and a well-defined team charter, you probably have more time to devote to building software, hands-on. You might also find yourself on the other side of this, when you're managing a large team, and on top of the people

management responsibilities you have a lot of strategy and alignment-related work, and these activities are higher leverage than writing code is.

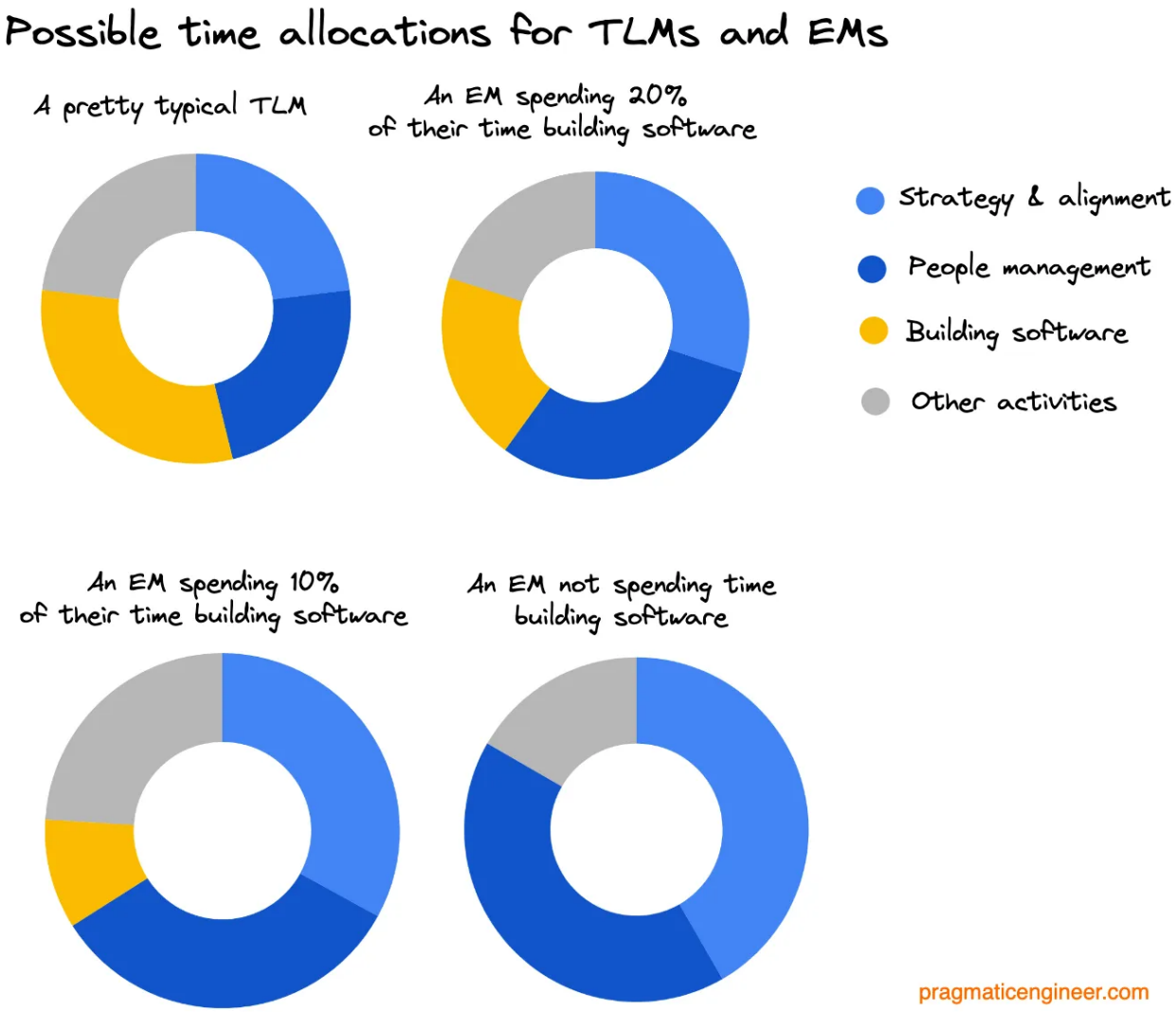
With some roles, you might have an explicit expectation of building software in some percentage of the time. For example, at Uber, the guidance for tech lead managers was that they should aim to be hands-on 50% of the time. Of course, that 50% was pretty aspirational, given the additional expectations these people had.

At Twitter, after Elon Musk took over, he mandated that all managers should code at least 20% of their time, all while managing 20 or more engineers, as we covered in The Scoop #31.

In all cases, something has to give. When you spend more time coding, you'll need to spend less time on one or more of:

- People management: such as helping people grow, giving them feedback, performance management
- Strategy and alignment: making sure the team is working on the right problems, and other teams and stakeholders are aligned; work will not need to be redone or thrown away
- Other activities: hiring, recruitment, mentoring, helping other teams with this or that.

Let's visualize some possible tradeoffs, depending on the amount of time spent being hands-on and building software:



Possible splits in allocating time, as a tech lead manager (TLM) or engineering manager (EM)

When deciding how much time to spend writing code - which is a subset of building software - do consider what activities you'll do less of, and if the allocation will be more, or less helpful for your team, and for yourself.

5. Beware of taking space from others

I remember a manager early in my career who had been recently promoted from senior engineer to engineering manager. He was very technically-minded and a productive backend software engineer. He would very frequently jump in and pick up tickets, code

with us, and do code reviews. He was a very hands-on manager. Great for the team. Right? Nope.

The backend engineers really did not like his approach.

He would jump into work which an engineer was midway through and introduce a new approach, sometimes taking over the task. He didn't have time to attend planning meetings as he had other stuff to do, but would announce we were to follow his approach, even though we'd collectively agreed on something else. He would do a code review with lots of comments, but not be around the next day when modifications were made, and then later complain if some modifications weren't as he expected.

However, there was one group that enjoyed working with this manager: the frontend engineers. This was because this manager had no frontend experience, so he left them to their own devices.

What was happening here was a senior engineer who was unable to let go of the technical work, to the point of micromanaging his team. If you're a manager or a lead, you'd hopefully think that you don't want to end up as a micromanager.

Be mindful of not snatching the "best" work from engineers on your team. "Best" might mean the most impactful, complex and interesting tasks. Managers who want to be hands-on can take work which other engineers really *want* to do, without noticing this.

If you pick up a challenging coding task, consider if that task would be a great challenge for a less experienced engineer, with you providing valuable advice and reviewing it. If you decide to lead a high-stakes project, perhaps it would be a fantastic stretch opportunity for a senior engineer to lead that project?

The professional growth of engineers on your team could be stunted if you take on the challenging, interesting or impactful work which they could do, and which you could meaningfully support. An engineer is unlikely to build a strong promotion case if you constantly lead complex and impactful projects, and they merely follow your plan when coding. Perhaps they would have a much stronger case if they did the planning, orchestration and a lot of the coding on the project?

If you decide to get hands-on, choose 'non-desirable' work, such as:

- Tasks nobody wants to do: cleaning up, improving tooling and build systems, oncall-related work.
- Work no one will do: tasks that are simply such low priority that the team won't get to them, like some bugs, investigating minor systems' health issues, and more.
- Work people fear doing: high-risk tasks or projects for which the team lacks the expertise or skills. This could be leading projects that are very high stakes or very complex for the skill sets and expertise of team members, or situations where it's the leader's job to represent the team. For example, if leadership is reviewing a high-impact outage and wants answers from your team, it could be a sensible approach for you to be in the 'hot seat' during this potentially stressful situation.

6. The reality of staying hands-on

The less hands-on work you do, the less hands-on you will be: this is the reality of seeking to stay "hands-on" as a manager. You can try to work around this, for example by working longer hours and spending that time on hands-on work, but it doesn't change the fact that when the demands of being a manager don't include much hands-on work, you simply can't be as hands-on as you were earlier in your career.

It's much easier to stay somewhat hands-on if you previously worked on the codebase, as an engineer. I do think many companies make a mistake by not onboarding engineering managers like software engineers to the codebase, instead rushing them into taking on managerial responsibilities.

Staff software engineer Jeethu Rao, formerly of Google and Meta, shared his observation on how Meta does it:

Jeethu Rao
@jeethu

@GergelyOrosz From what I saw at FB and Google, even people hired as EMs and Directors, spent the first 6 months as ICs before they began managing. At

FB, they used to say that the blast radius of a bad EM hire is much bigger than that of a bad IC hire, and thus, the abundance of caution.

1:31 PM · Jan 30, 2023

145 Likes **1** Retweet

Although I'm skeptical about the 6 months detail, Meta very deliberately onboards engineering managers the same way as software engineers. It's much easier to make good decisions that impact the team when you know how your team works, inside out.

I was a principal engineer and tech lead at Skyscanner before joining Uber. When I got the offer to join, I was hoping for a position as a tech lead, as I had led a team before. However, Uber offered a Senior Engineer role, which I took as I figured I'd have enough new things to learn.

Joining as a software engineer was a great decision, in my case. I transitioned to become an engineering manager 6 months later, and that half year of working on the codebase side-by-side with engineers helped me to stay hands-on, involved, and very technical throughout my time at Uber. I later observed that engineering managers who joined and didn't have time to work on the codebase struggled more to gain the trust of their teams and to understand the systems.

If your goal is to stay hands-on and do more technical work, you will likely be disappointed when you go above the line manager level. That's because hands-on work is simply not how you create impact. If you struggle with this realization, there's always the option of deciding to step down from the managing of managers, like Principal engineering manager Bill Ticehurst at Microsoft did:

Bill Ticehurst
@billticehurst

@GergelyOrosz I've gone back from being a 2nd level manager to a 1st level manager a couple times now because I missed the technical work as an M2. (I still like management though, so haven't gone back to individual contributor).

2:50 AM · Jan 30, 2023

Takeaways

In this issue, we've covered a variety of approaches on how to stay hands-on, as an engineering manager.

Onboard to your team or company like a software engineer does. If you do, it becomes several times easier to be hands-on and boosts your involvement. Managers usually face the temptation to skip engineer onboarding: avoid this temptation! If you are an engineering leader at your company, consider making it mandatory that new managers also go through engineer onboarding.

There's a host of activities that can help you stay hands-on, including:

- Participating in design documents / RFCs / ERDs / ADRs
- Doing code reviews
- Facilitating or participating in operational reviews
- Going oncall
- Following up on outages, and owning incident reviews

The most obvious way to stay hands-on is to write code. However, this is usually the most challenging task for engineering leaders, as writing code requires uninterrupted time, which is something in short supply. So, consider picking up small and non-critical tasks, pair programming, prototyping or code katas, in order to not hold up your team when the inevitable interruptions come.

Beware of being that manager who unintentionally removes growth opportunities from their team, in an effort to stay hands-on. When you consider picking up work, ask these questions:

- Is this work impactful?
- Is it important?
- Is it interesting?

If the answer is “yes” to any of these, consider whether it’s *you* who needs to do the work? Would it not be a better opportunity for an engineer on your team to pick up the task, with you supporting them? Be especially conscious when it comes to impactful work, which could help engineers to grow professionally, get promoted, and claim the impact can help during performance calibrations.

The reality of staying hands-on is that the further you are from the code, the harder it is to do. As a line manager, there are plenty of things you can do to stay hands-on. However, once you become a manager of managers, you need to accept that opportunities to be hands-on will dramatically shrink. It’s a natural part of your career growth.

The good news is that even when you don’t have time for hands-on work, you still have plenty of ways to stay technical. In a follow-up issue, we’ll dig into how to stay technical as an engineering manager.

118 Likes

6 Comments

Write a comment...

Dusko Bajic Writes Crispy Engineering Feb 14

I really like this write up as it emphasize the "support" role of an EM one way or another. In my opinion that's what differentiates EM from other type of managers in software (or other fields) is that you're there to support your team. Taking "boring" tasks or taking on-call rota is +1 to that.

Anyhow, on the contrary to staying technical, I think it's important to mention that EM's (first timers) are exception of this rule. Because Engineering Manager that is in this role for the first time, has to spend lot's of time to learn how to be a good EM for their teams. In order to do that, I think spending time on education, self-improvement and realization of what this role really is, is more valuable than staying technical.

Technology won't go anyway and I feel there is some threshold where EM should go from 100% managerial work to 80% managerial and 20% technical (whatever the ratio), after some time. Maybe this is an edge case when EM is made through internal-promotion, rather then external hiring.

Interested in your opinion 🙏 . Thanks!

LIKE (5) REPLY

1 reply

Jude Clermont Feb 14

Thanks for your article - you have given me enough information for me to decide to stay technical (Hands-on) rather than Tech Lead or Manager. As I cannot find the time to do coding, while being stuck in boring meetings and having to spoon feed other managers how they should technically conduct themselves since they lack the understanding of the technology that their department is responsible for. I found myself blocking my team by trying to take on certain task and not being available when I made suggestions due to being tied up in meetings. I have now accepted a role as an Engineer with another firm.

thank you for your article

LIKE (4) REPLY

4 more comments...